

PRACTICA – SEMANA 11

INTEGRANTES:

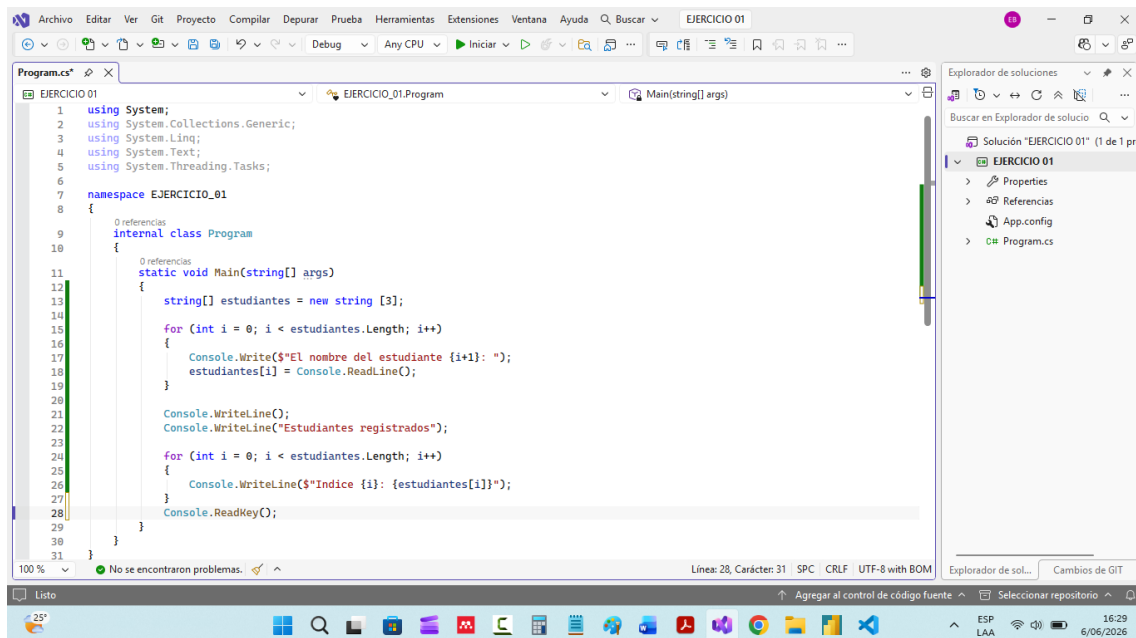
YANILKA CARLA CASTRO DELGADO

EDGAR GENARO PARI BENITES

Ejercicio 1. Registro de estudiantes inscritos

Un taller necesita registrar los nombres de 3 estudiantes inscritos. El programa debe almacenar cada nombre en un arreglo unidimensional y, al finalizar el registro, mostrar todos los nombres junto con el índice donde fueron guardados.

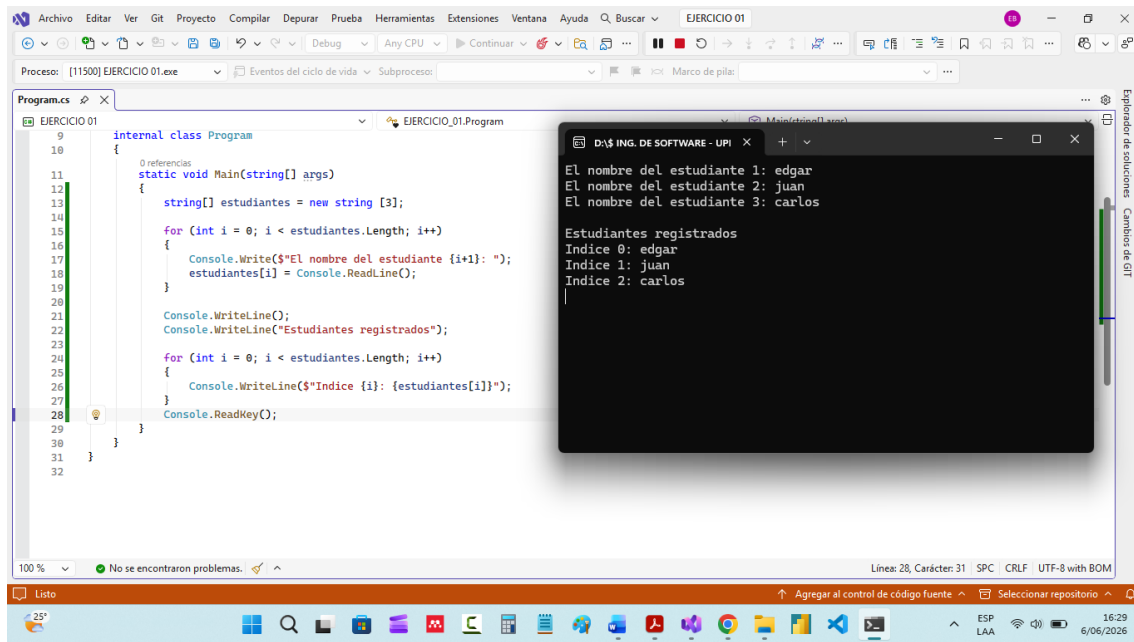
C#



```
1 using System;
2 using System.Collections.Generic;
3 using System.Linq;
4 using System.Text;
5 using System.Threading.Tasks;
6
7 namespace EJERCICIO_01
8 {
9     internal class Program
10     {
11         static void Main(string[] args)
12         {
13             string[] estudiantes = new string [3];
14
15             for (int i = 0; i < estudiantes.Length; i++)
16             {
17                 Console.WriteLine($"El nombre del estudiante {i+1}: ");
18                 estudiantes[i] = Console.ReadLine();
19             }
20
21             Console.WriteLine();
22             Console.WriteLine("Estudiantes registrados");
23
24             for (int i = 0; i < estudiantes.Length; i++)
25             {
26                 Console.WriteLine($"Índice {i}: {estudiantes[i]}");
27             }
28             Console.ReadKey();
29         }
30     }
31 }
```

The screenshot shows the Visual Studio IDE with a C# project named "EJERCICIO_01". The code in Program.cs registers three students by reading input from the console and then displays the list with their indices. The output window shows the following text:

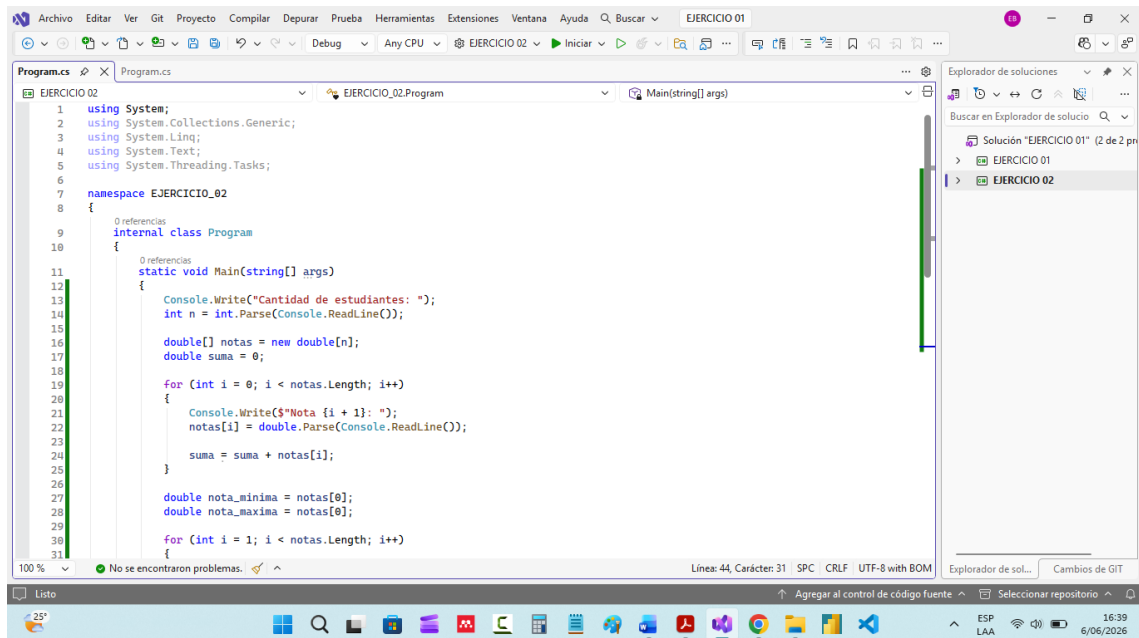
```
El nombre del estudiante 1:
El nombre del estudiante 2:
El nombre del estudiante 3:
Estudiantes registrados
Índice 0:
Índice 1:
Índice 2:
```



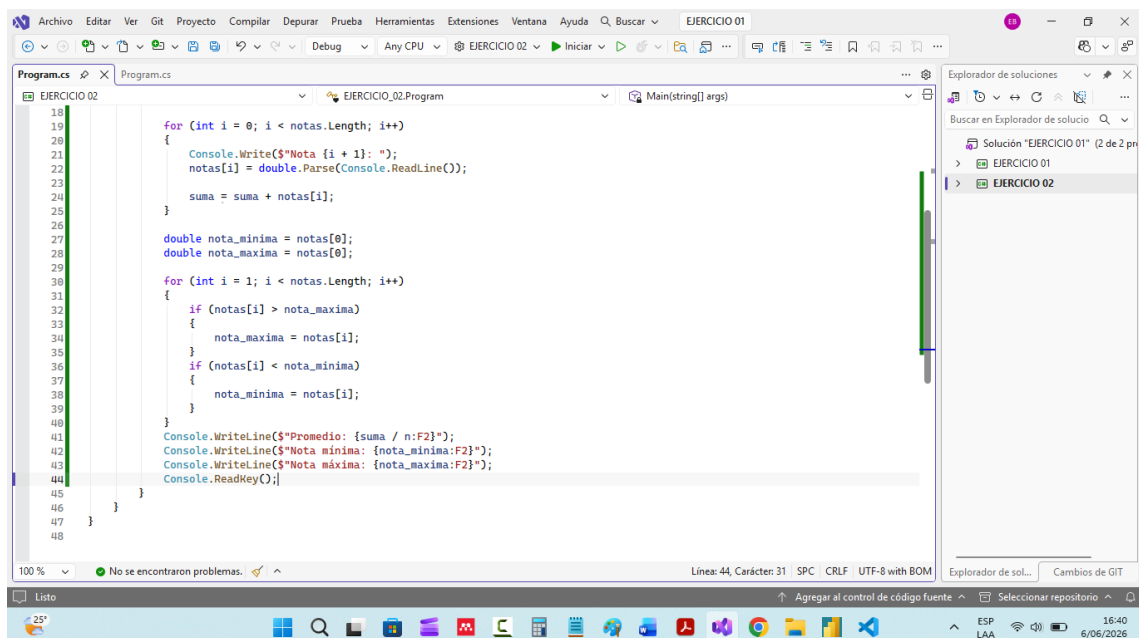
Ejercicio 2. Notas de estudiantes: promedio, mínima y máxima

Un docente necesita registrar las notas de N estudiantes. El programa debe almacenar las notas en un arreglo unidimensional, calcular el promedio general, identificar la nota mínima y la nota máxima, y mostrar los resultados con dos decimales.

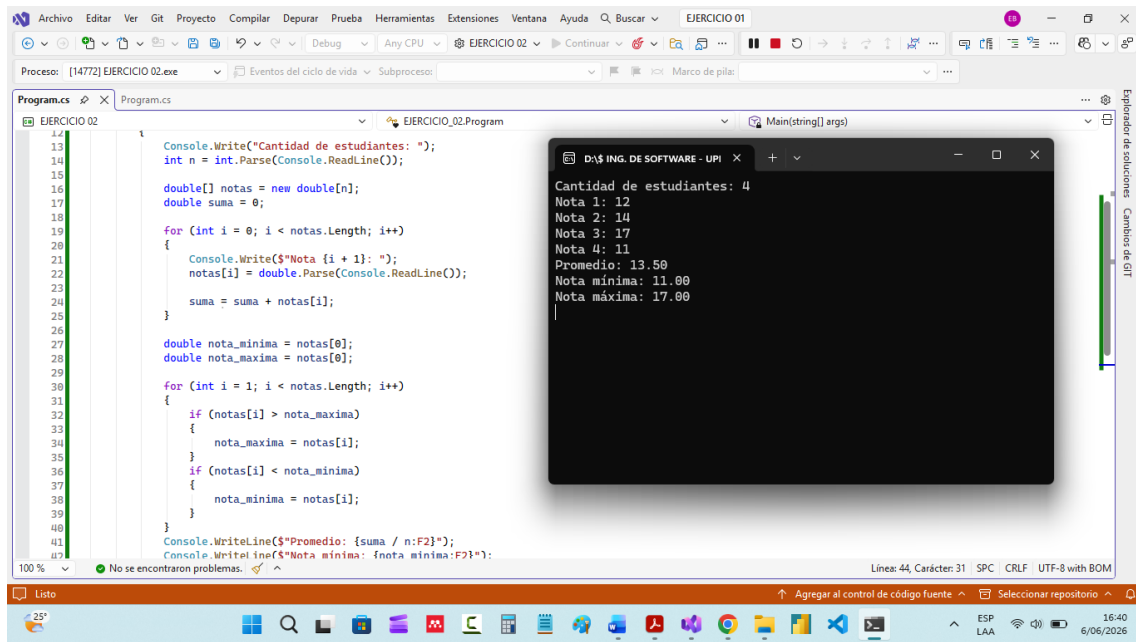
C#



```
1 using System;
2 using System.Collections.Generic;
3 using System.Linq;
4 using System.Text;
5 using System.Threading.Tasks;
6
7 namespace EJERCICIO_02
8 {
9     internal class Program
10     {
11         static void Main(string[] args)
12         {
13             Console.WriteLine("Cantidad de estudiantes: ");
14             int n = int.Parse(Console.ReadLine());
15
16             double[] notas = new double[n];
17             double suma = 0;
18
19             for (int i = 0; i < notas.Length; i++)
20             {
21                 Console.WriteLine($"Nota {i + 1}: ");
22                 notas[i] = double.Parse(Console.ReadLine());
23
24                 suma = suma + notas[i];
25             }
26
27             double nota_minima = notas[0];
28             double nota_maxima = notas[0];
29
30             for (int i = 1; i < notas.Length; i++)
31             {
```



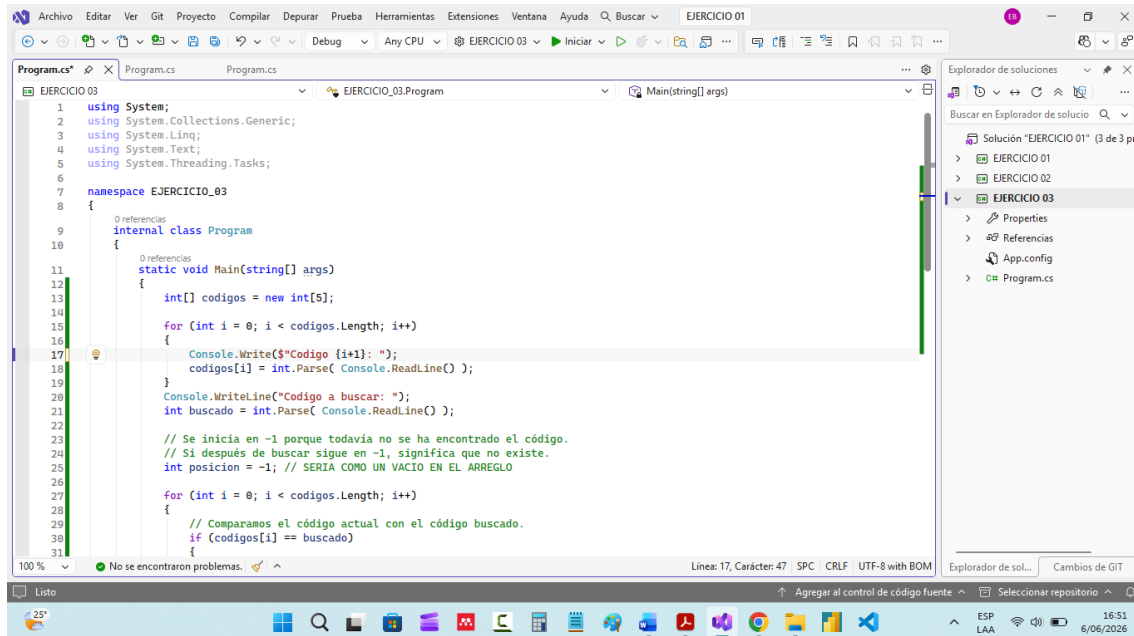
```
32             {
33                 if (notas[i] > nota_maxima)
34                 {
35                     nota_maxima = notas[i];
36                 }
37                 if (notas[i] < nota_minima)
38                 {
39                     nota_minima = notas[i];
40                 }
41             }
42             Console.WriteLine($"Promedio: {suma / n:F2}");
43             Console.WriteLine($"Nota mínima: {nota_minima:F2}");
44             Console.WriteLine($"Nota máxima: {nota_maxima:F2}");
45             Console.ReadKey();
46         }
47     }
48 }
```



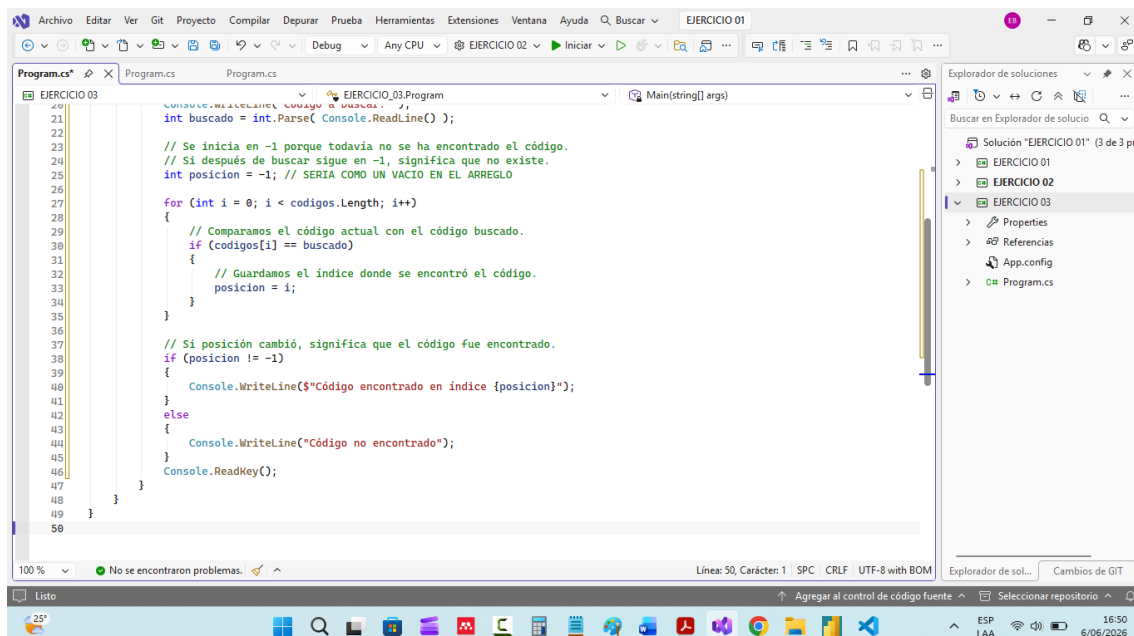
Ejercicio 3. Buscar código de producto en inventario

Una tienda registra 5 códigos de productos en un arreglo unidimensional. Luego solicita un código a buscar. El programa debe recorrer el arreglo y mostrar si el código existe, indicando el índice donde fue encontrado. Si no existe, debe mostrar "Código no encontrado".

C#



```
1 using System;
2 using System.Collections.Generic;
3 using System.Linq;
4 using System.Text;
5 using System.Threading.Tasks;
6
7 namespace EJERCICIO_03
8 {
9     internal class Program
10     {
11         static void Main(string[] args)
12         {
13             int[] codigos = new int[5];
14             for (int i = 0; i < codigos.Length; i++)
15             {
16                 Console.WriteLine($"Código {i+1}: ");
17                 codigos[i] = int.Parse( Console.ReadLine() );
18             }
19             Console.WriteLine("Código a buscar: ");
20             int buscado = int.Parse( Console.ReadLine() );
21
22             // Se inicia en -1 porque todavía no se ha encontrado el código.
23             // Si después de buscar sigue en -1, significa que no existe.
24             int posicion = -1; // SERIA COMO UN VACIO EN EL ARREGLO
25
26             for (int i = 0; i < codigos.Length; i++)
27             {
28                 // Comparamos el código actual con el código buscado.
29                 if (codigos[i] == buscado)
30                 {
31                     // Se encontró el código, guardamos la posición.
32                     posicion = i;
33                 }
34             }
35
36             // Si posición cambió, significa que el código fue encontrado.
37             if (posicion != -1)
38             {
39                 Console.WriteLine($"Código encontrado en índice {posicion}");
40             }
41             else
42             {
43                 Console.WriteLine("Código no encontrado");
44             }
45             Console.ReadKey();
46         }
47     }
48 }
49
50
```



```
1 using System;
2 using System.Collections.Generic;
3 using System.Linq;
4 using System.Text;
5 using System.Threading.Tasks;
6
7 namespace EJERCICIO_03
8 {
9     internal class Program
10     {
11         static void Main(string[] args)
12         {
13             int[] codigos = new int[5];
14             for (int i = 0; i < codigos.Length; i++)
15             {
16                 Console.WriteLine($"Código {i+1}: ");
17                 codigos[i] = int.Parse( Console.ReadLine() );
18             }
19             Console.WriteLine("Código a buscar: ");
20             int buscado = int.Parse( Console.ReadLine() );
21
22             // Se inicia en -1 porque todavía no se ha encontrado el código.
23             // Si después de buscar sigue en -1, significa que no existe.
24             int posicion = -1; // SERIA COMO UN VACIO EN EL ARREGLO
25
26             for (int i = 0; i < codigos.Length; i++)
27             {
28                 // Comparamos el código actual con el código buscado.
29                 if (codigos[i] == buscado)
30                 {
31                     // Guardamos el índice donde se encontró el código.
32                     posicion = i;
33                 }
34             }
35
36             // Si posición cambió, significa que el código fue encontrado.
37             if (posicion != -1)
38             {
39                 Console.WriteLine($"Código encontrado en índice {posicion}");
40             }
41             else
42             {
43                 Console.WriteLine("Código no encontrado");
44             }
45             Console.ReadKey();
46         }
47     }
48 }
49
50
```

Visual Studio interface showing a C# program named EJERCICIO_03. The code defines an internal class Program with a Main method. It reads five integers into an array and then searches for a target integer. The console output shows the five numbers and the target, followed by the message "Código encontrado en índice 2".

```
1 using System;
2 using System.Collections.Generic;
3 using System.Linq;
4 using System.Text;
5 using System.Threading.Tasks;
6
7 namespace EJERCICIO_03
8 {
9     internal class Program
10     {
11         static void Main(string[] args)
12         {
13             int[] codigos = new int[5];
14
15             for (int i = 0; i < codigos.Length; i++)
16             {
17                 Console.WriteLine($"Codigo {i+1}: ");
18                 codigos[i] = int.Parse( Console.ReadLine() );
19             }
20             Console.WriteLine("Codigo a buscar: ");
21             int buscado = int.Parse( Console.ReadLine() );
22
23             // Se inicia en -1 porque todavía no se ha encontrado el código.
24             // Si después de buscar sigue en -1, significa que no existe.
25             int posicion = -1; // SERIA COMO UN VACIO EN EL ARREGLO
26
27             for (int i = 0; i < codigos.Length; i++)
28             {
29                 // Comparamos el código actual con el código buscado.
30             }
31         }
32     }
33 }
```

Console Output:

```
Codigo 1: 10
Codigo 2: 20
Codigo 3: 30
Codigo 4: 40
Codigo 5: 50
Codigo a buscar:
30
Código encontrado en índice 2
```

Visual Studio interface showing the same C# program as above. The code is identical, but the search logic is updated to check if the target is found. The console output shows the same numbers and target, but the message is "Código no encontrado".

```
11 static void Main(string[] args)
12 {
13     int[] codigos = new int[5];
14
15     for (int i = 0; i < codigos.Length; i++)
16     {
17         Console.WriteLine($"Codigo {i+1}: ");
18         codigos[i] = int.Parse( Console.ReadLine() );
19     }
20     Console.WriteLine("Codigo a buscar: ");
21     int buscado = int.Parse( Console.ReadLine() );
22
23     // Se inicia en -1 porque todavía no se ha encontrado el código.
24     // Si después de buscar sigue en -1, significa que no existe.
25     int posicion = -1; // SERIA COMO UN VACIO EN EL ARREGLO
26
27     for (int i = 0; i < codigos.Length; i++)
28     {
29         // Comparamos el código actual con el código buscado.
30         if (codigos[i] == buscado)
31         {
32             // Guardamos el índice donde se encontró el código.
33             posicion = i;
34         }
35     }
36
37     // Si posición cambió, significa que el código fue encontrado.
38     if (posicion != -1)
39     {
40         Console.WriteLine($"Código encontrado en índice {posicion}");
41     }
42 }
```

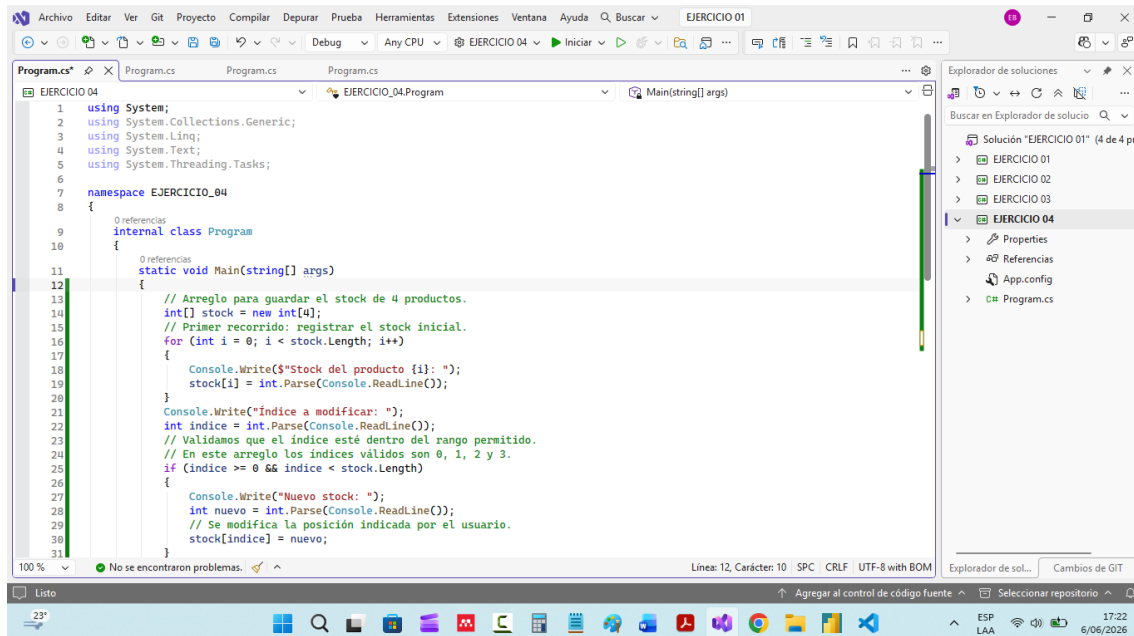
Console Output:

```
Codigo 1: 10
Codigo 2: 20
Codigo 3: 30
Codigo 4: 40
Codigo 5: 50
Codigo a buscar:
99
Código no encontrado
```

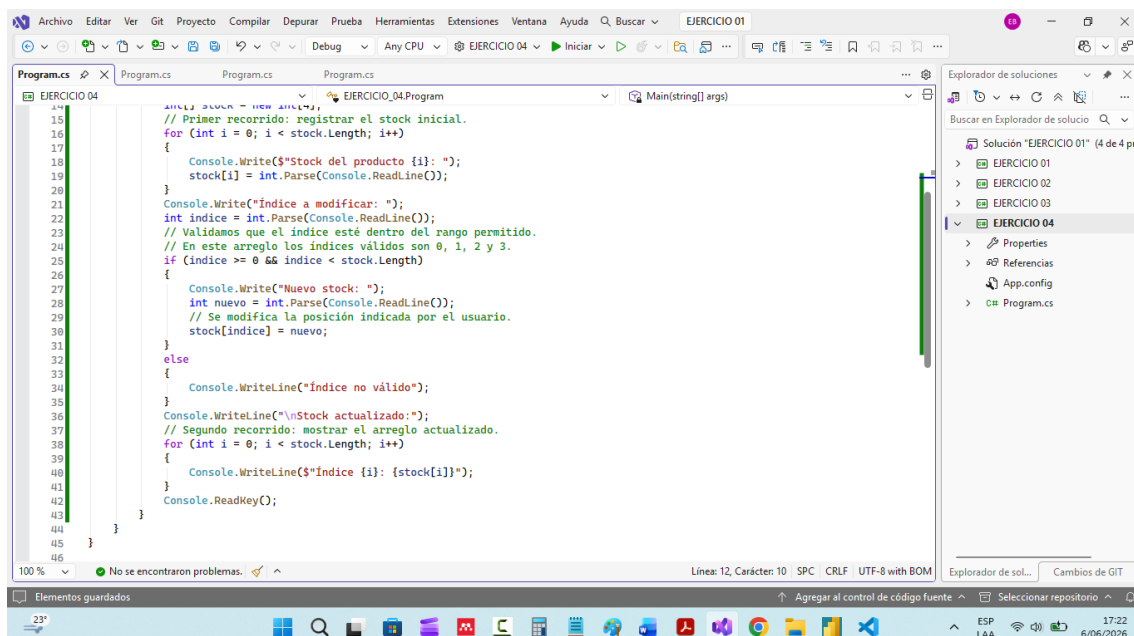
Ejercicio 4. Modificar stock por índice

Un almacén tiene registrado el stock de 4 productos en un arreglo unidimensional. El usuario debe ingresar el índice del producto que desea modificar y luego el nuevo stock. El programa debe validar que el índice exista antes de realizar la modificación. Finalmente, debe mostrar el stock actualizado de todos los productos.

C#



```
1 using System;
2 using System.Collections.Generic;
3 using System.Linq;
4 using System.Text;
5 using System.Threading.Tasks;
6
7 namespace EJERCICIO_04
8 {
9     internal class Program
10     {
11         static void Main(string[] args)
12         {
13             // Arreglo para guardar el stock de 4 productos.
14             int[] stock = new int[4];
15             // Primer recorrido: registrar el stock inicial.
16             for (int i = 0; i < stock.Length; i++)
17             {
18                 Console.WriteLine($"Stock del producto {i}: ");
19                 stock[i] = int.Parse(Console.ReadLine());
20             }
21             Console.WriteLine("Índice a modificar: ");
22             int indice = int.Parse(Console.ReadLine());
23             // Validamos que el índice esté dentro del rango permitido.
24             // En este arreglo los índices válidos son 0, 1, 2 y 3.
25             if (indice >= 0 && indice < stock.Length)
26             {
27                 Console.WriteLine("Nuevo stock: ");
28                 int nuevo = int.Parse(Console.ReadLine());
29                 // Se modifica la posición indicada por el usuario.
30                 stock[indice] = nuevo;
31             }
```



```
14 int[] stock = new int[4];
15 // Primer recorrido: registrar el stock inicial.
16 for (int i = 0; i < stock.Length; i++)
17 {
18     Console.WriteLine($"Stock del producto {i}: ");
19     stock[i] = int.Parse(Console.ReadLine());
20 }
21 Console.WriteLine("Índice a modificar: ");
22 int indice = int.Parse(Console.ReadLine());
23 // Validamos que el índice esté dentro del rango permitido.
24 // En este arreglo los índices válidos son 0, 1, 2 y 3.
25 if (indice >= 0 && indice < stock.Length)
26 {
27     Console.WriteLine("Nuevo stock: ");
28     int nuevo = int.Parse(Console.ReadLine());
29     // Se modifica la posición indicada por el usuario.
30     stock[indice] = nuevo;
31 }
32 else
33 {
34     Console.WriteLine("Índice no válido");
35 }
36 Console.WriteLine("\nStock actualizado:");
37 // Segundo recorrido: mostrar el arreglo actualizado.
38 for (int i = 0; i < stock.Length; i++)
39 {
40     Console.WriteLine($"Índice {i}: {stock[i]}");
41 }
42 Console.ReadKey();
43 }
44 }
45 }
46 }
```

```
14  int[] stock = new int[4];
15  // Primer recorrido: registrar el stock inicial.
16  for (int i = 0; i < stock.Length; i++)
17  {
18      Console.WriteLine($"Stock del producto {i}: ");
19      stock[i] = int.Parse(Console.ReadLine());
20  }
21  Console.WriteLine("Índice a modificar: ");
22  int indice = int.Parse(Console.ReadLine());
23  // Validamos que el índice esté dentro del rango permitido.
24  // En este arreglo los índices válidos son 0, 1, 2 y 3.
25  if (indice >= 0 && indice < stock.Length)
26  {
27      Console.WriteLine("Nuevo stock: ");
28      int nuevo = int.Parse(Console.ReadLine());
29      // Se modifica la posición indicada por el usuario.
30      stock[indice] = nuevo;
31  }
32  else
33  {
34      Console.WriteLine("Índice no válido");
35  }
36  Console.WriteLine("\nStock actualizado:");
37  // Segundo recorrido: mostrar el arreglo actualizado.
38  for (int i = 0; i < stock.Length; i++)
39  {
40      Console.WriteLine($"Índice {i}: {stock[i]}");
41  }
42  Console.ReadKey();
43  }
```

Stock del producto 0: 5
Stock del producto 1: 8
Stock del producto 2: 3
Stock del producto 3: 9
Índice a modificar: 2
Nuevo stock: 10

Stock actualizado:
Índice 0: 5
Índice 1: 8
Índice 2: 10
Índice 3: 9

```
14  int[] stock = new int[4];
15  // Primer recorrido: registrar el stock inicial.
16  for (int i = 0; i < stock.Length; i++)
17  {
18      Console.WriteLine($"Stock del producto {i}: ");
19      stock[i] = int.Parse(Console.ReadLine());
20  }
21  Console.WriteLine("Índice a modificar: ");
22  int indice = int.Parse(Console.ReadLine());
23  // Validamos que el índice esté dentro del rango permitido.
24  // En este arreglo los índices válidos son 0, 1, 2 y 3.
25  if (indice >= 0 && indice < stock.Length)
26  {
27      Console.WriteLine("Nuevo stock: ");
28      int nuevo = int.Parse(Console.ReadLine());
29      // Se modifica la posición indicada por el usuario.
30      stock[indice] = nuevo;
31  }
32  else
33  {
34      Console.WriteLine("Índice no válido");
35  }
36  Console.WriteLine("\nStock actualizado:");
37  // Segundo recorrido: mostrar el arreglo actualizado.
38  for (int i = 0; i < stock.Length; i++)
39  {
40      Console.WriteLine($"Índice {i}: {stock[i]}");
41  }
42  Console.ReadKey();
43  }
```

Stock del producto 0: 5
Stock del producto 1: 8
Stock del producto 2: 3
Stock del producto 3: 9
Índice a modificar: 7
Índice no válido

Stock actualizado:
Índice 0: 5
Índice 1: 8
Índice 2: 3
Índice 3: 9

Ejercicio 5. Buscar primera temperatura mayor a 30 °C

Se registran las temperaturas de 7 días en un arreglo unidimensional. El programa debe identificar el primer día en que la temperatura superó los 30 °C. Si ninguna temperatura supera los 30 °C, debe mostrar que no hubo alerta.

C#

```
1 using System;
2 using System.Collections.Generic;
3 using System.Linq;
4 using System.Text;
5 using System.Threading.Tasks;
6
7 namespace EJERCICIO_05
8 {
9     internal class Program
10     {
11         static void Main(string[] args)
12         {
13             double[] temperaturas = new double[7];
14             int dia_encontrado = -1;
15
16             for (int i = 0; i < temperaturas.Length; i++)
17             {
18                 Console.WriteLine($"INGRESE LA TEMPERATURA DEL DIA {i+1}: ");
19                 temperaturas[i] = double.Parse(Console.ReadLine());
20             }
21
22             for (int i = 0; i < temperaturas.Length; i++)
23             {
24                 if (temperaturas[i] > 30 && dia_encontrado == -1)
25                 {
26                     dia_encontrado = i;
27                 }
28             }
29
30             if (dia_encontrado != -1)
31             {
32                 Console.WriteLine($"Primera alerta del dia, {dia_encontrado + 1}");
33             }
34         }
35     }
36 }
```

```
14 static void Main(string[] args)
15 {
16     double[] temperaturas = new double[7];
17     int dia_encontrado = -1;
18
19     for (int i = 0; i < temperaturas.Length; i++)
20     {
21         Console.WriteLine($"INGRESE LA TEMPERATURA DEL DIA {i+1}: ");
22         temperaturas[i] = double.Parse(Console.ReadLine());
23     }
24
25     for (int i = 0; i < temperaturas.Length; i++)
26     {
27         if (temperaturas[i] > 30 && dia_encontrado == -1)
28         {
29             dia_encontrado = i;
30         }
31     }
32
33     if (dia_encontrado != -1)
34     {
35         Console.WriteLine($"Primera alerta del dia, {dia_encontrado + 1}");
36     }
37     else
38     {
39         Console.WriteLine("No hubo alerta de temperatura");
40     }
41     Console.ReadKey();
42 }
```

```
12 {
13     double[] temperaturas = new double[7];
14     int dia_encontrado = -1;
15
16     for (int i = 0; i < temperaturas.Length; i++)
17     {
18         Console.Write($"INGRESE LA TEMPERATURA DEL DIA {i+1}: ");
19         temperaturas[i] = double.Parse(Console.ReadLine());
20     }
21
22     for (int i = 0; i < temperaturas.Length; i++)
23     {
24         if (temperaturas[i] > 30 && dia_encontrado == -1)
25         {
26             dia_encontrado = i;
27         }
28     }
29     if (dia_encontrado != -1)
30     {
31         Console.WriteLine($"Primera alerta del día, {dia_encontrado + 1}");
32     }
33     else
34     {
35         Console.WriteLine("No hubo alerta de temperatura");
36     }
37     Console.ReadKey();
38 }
39 }
40
41 }
```

INGRESE LA TEMPERATURA DEL DIA 1: 25
INGRESE LA TEMPERATURA DEL DIA 2: 28
INGRESE LA TEMPERATURA DEL DIA 3: 31
INGRESE LA TEMPERATURA DEL DIA 4: 29
INGRESE LA TEMPERATURA DEL DIA 5: 32
INGRESE LA TEMPERATURA DEL DIA 6: 27
INGRESE LA TEMPERATURA DEL DIA 7: 26
Primera alerta del día, 3

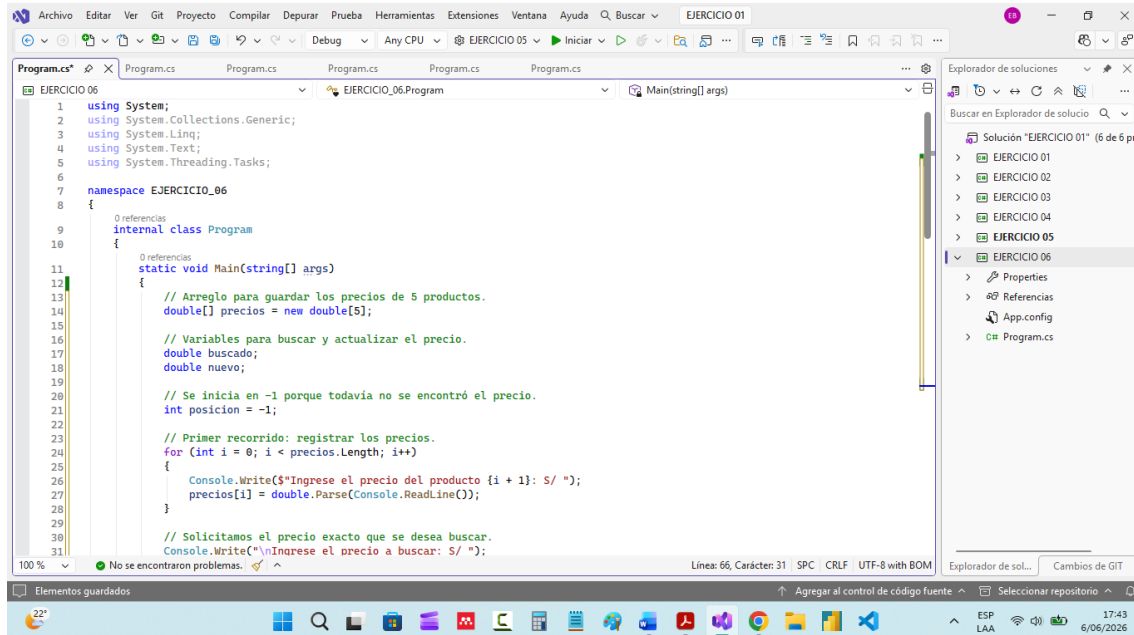
```
11 {
12     double[] temperaturas = new double[7];
13     int dia_encontrado = -1;
14
15     for (int i = 0; i < temperaturas.Length; i++)
16     {
17         Console.Write($"INGRESE LA TEMPERATURA DEL DIA {i+1}: ");
18         temperaturas[i] = double.Parse(Console.ReadLine());
19     }
20
21     for (int i = 0; i < temperaturas.Length; i++)
22     {
23         if (temperaturas[i] > 30 && dia_encontrado == -1)
24         {
25             dia_encontrado = i;
26         }
27     }
28     if (dia_encontrado != -1)
29     {
30         Console.WriteLine($"Primera alerta del día, {dia_encontrado + 1}");
31     }
32     else
33     {
34         Console.WriteLine("No hubo alerta de temperatura");
35     }
36     Console.ReadKey();
37 }
38 }
```

INGRESE LA TEMPERATURA DEL DIA 1: 24
INGRESE LA TEMPERATURA DEL DIA 2: 25
INGRESE LA TEMPERATURA DEL DIA 3: 26
INGRESE LA TEMPERATURA DEL DIA 4: 27
INGRESE LA TEMPERATURA DEL DIA 5: 28
INGRESE LA TEMPERATURA DEL DIA 6: 29
INGRESE LA TEMPERATURA DEL DIA 7: 30
No hubo alerta de temperatura

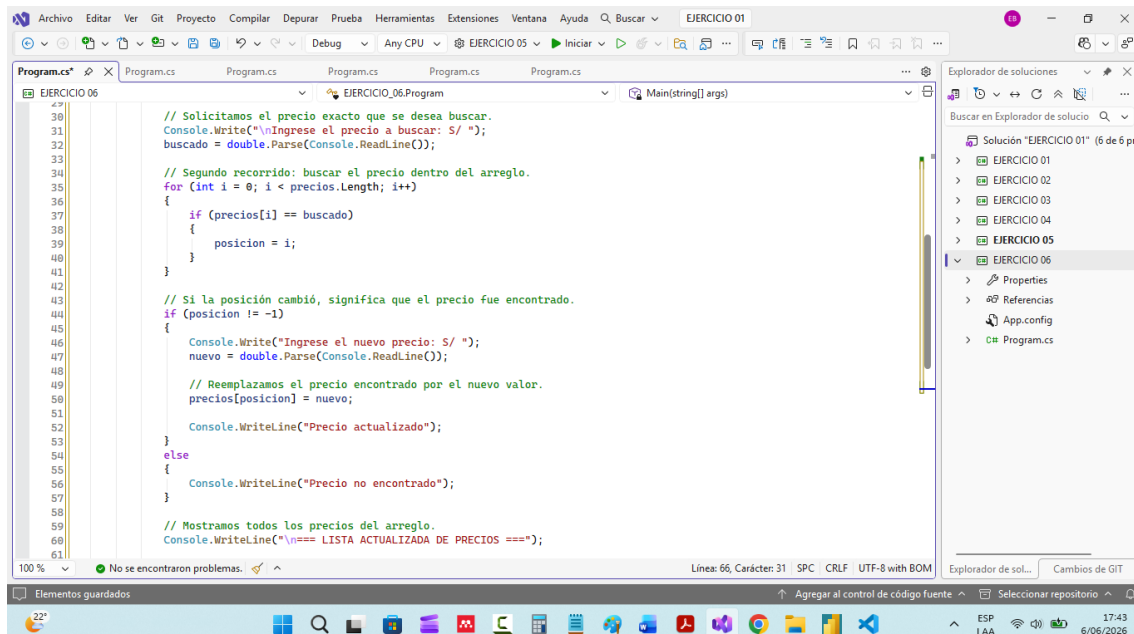
Ejercicio 6. Buscar y modificar precio de producto

Se registran los precios de 5 productos en un arreglo unidimensional. Luego, el usuario ingresa un precio exacto a buscar. Si el precio existe en el arreglo, el programa debe solicitar un nuevo precio y reemplazar el valor encontrado. Si no existe, debe mostrar "Precio no encontrado".

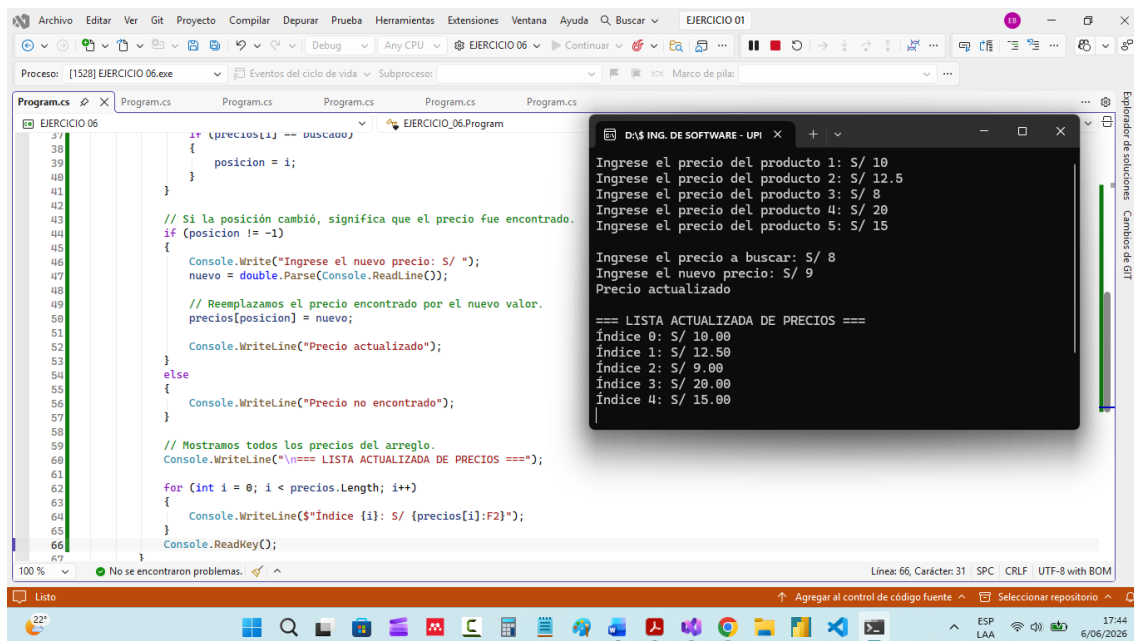
C#



```
1 using System;
2 using System.Collections.Generic;
3 using System.Linq;
4 using System.Text;
5 using System.Threading.Tasks;
6
7 namespace EJERCICIO_06
8 {
9     0 referencias
10     internal class Program
11     {
12         0 referencias
13         static void Main(string[] args)
14         {
15             // Arreglo para guardar los precios de 5 productos.
16             double[] precios = new double[5];
17
18             // Variables para buscar y actualizar el precio.
19             double buscado;
20             double nuevo;
21
22             // Se inicia en -1 porque todavía no se encontró el precio.
23             int posicion = -1;
24
25             // Primer recorrido: registrar los precios.
26             for (int i = 0; i < precios.Length; i++)
27             {
28                 Console.WriteLine($"Ingrese el precio del producto {i + 1}: $/ ");
29                 precios[i] = double.Parse(Console.ReadLine());
30             }
31
32             // Solicitamos el precio exacto que se desea buscar.
33             Console.WriteLine($"Ingrese el precio a buscar: $/ ");
```



```
34 // Solicitamos el precio exacto que se desea buscar.
35 Console.WriteLine($"Ingrese el precio a buscar: $/ ");
36 buscado = double.Parse(Console.ReadLine());
37
38 // Segundo recorrido: buscar el precio dentro del arreglo.
39 for (int i = 0; i < precios.Length; i++)
40 {
41     if (precios[i] == buscado)
42     {
43         posicion = i;
44     }
45 }
46
47 // Si la posición cambió, significa que el precio fue encontrado.
48 if (posicion != -1)
49 {
50     Console.WriteLine("Ingrese el nuevo precio: $/ ");
51     nuevo = double.Parse(Console.ReadLine());
52
53     // Reemplazamos el precio encontrado por el nuevo valor.
54     precios[posicion] = nuevo;
55
56     Console.WriteLine("Precio actualizado");
57 }
58 else
59 {
60     Console.WriteLine("Precio no encontrado");
61 }
62
63 // Mostramos todos los precios del arreglo.
64 Console.WriteLine($"LISTA ACTUALIZADA DE PRECIOS ==");
```



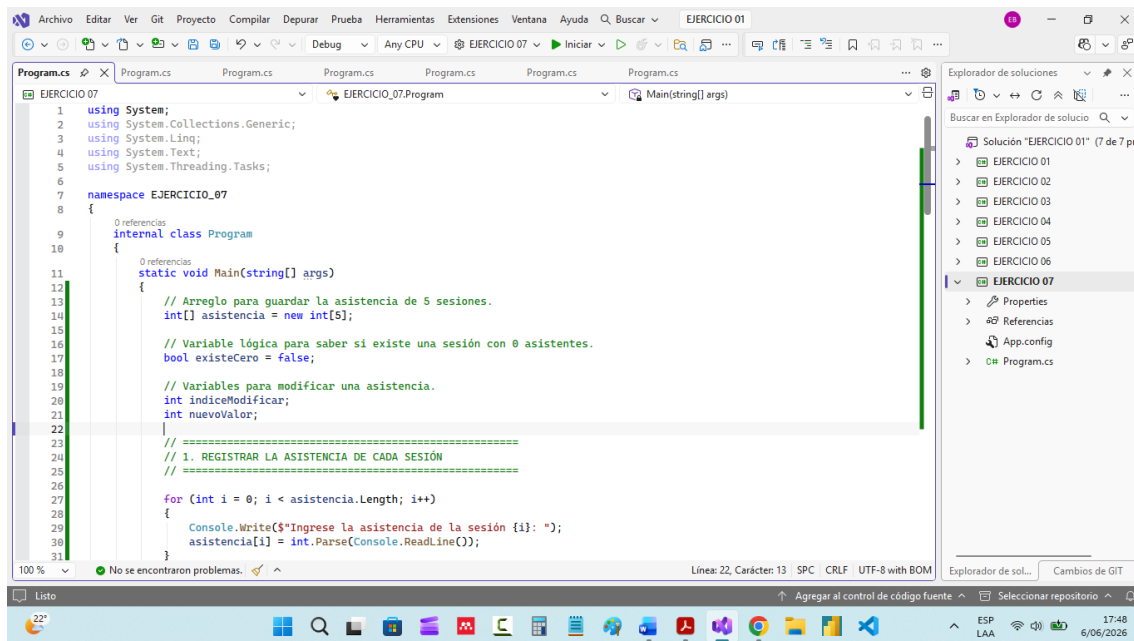
Ejercicio 7. Control de asistencia por sesión

Un docente registra la asistencia de **5 sesiones** de un taller en un arreglo unidimensional. El programa debe:

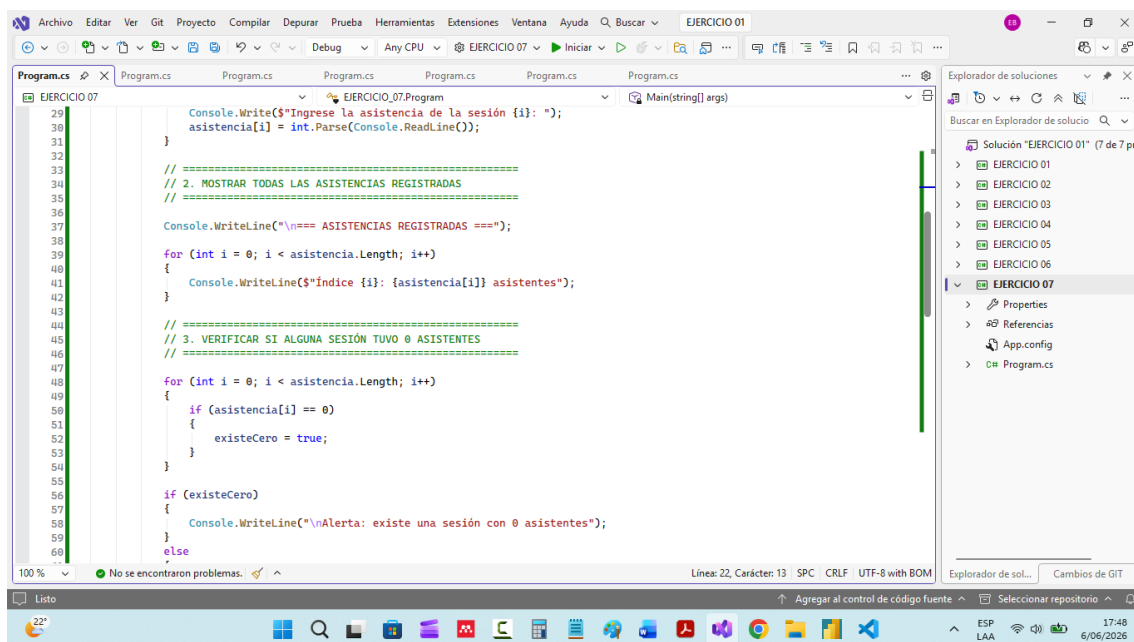
1. Registrar la asistencia de cada sesión.
2. Mostrar todas las asistencias registradas.
3. Verificar si alguna sesión tuvo exactamente **0 asistentes**.
4. Permitir modificar una asistencia indicando su índice.
5. Mostrar las asistencias actualizadas.

Si el índice ingresado no es válido, debe mostrar el mensaje: Índice no válido

C#



```
1 using System;
2 using System.Collections.Generic;
3 using System.Linq;
4 using System.Text;
5 using System.Threading.Tasks;
6
7 namespace EJERCICIO_07
8 {
9     internal class Program
10     {
11         static void Main(string[] args)
12         {
13             // Arreglo para guardar la asistencia de 5 sesiones.
14             int[] asistencia = new int[5];
15
16             // Variable lógica para saber si existe una sesión con 0 asistentes.
17             bool existeCero = false;
18
19             // Variables para modificar una asistencia.
20             int indiceModificar;
21             int nuevoValor;
22
23             // =====
24             // 1. REGISTRAR LA ASISTENCIA DE CADA SESIÓN
25             // =====
26
27             for (int i = 0; i < asistencia.Length; i++)
28             {
29                 Console.WriteLine($"Ingrese la asistencia de la sesión {i}: ");
30                 asistencia[i] = int.Parse(Console.ReadLine());
31             }
```



```
32
33
34             // =====
35             // 2. MOSTRAR TODAS LAS ASISTENCIAS REGISTRADAS
36             // =====
37
38             Console.WriteLine("\n=== ASISTENCIAS REGISTRADAS ===");
39
40             for (int i = 0; i < asistencia.Length; i++)
41             {
42                 Console.WriteLine($"Índice {i}: {asistencia[i]} asistentes");
43             }
44
45             // =====
46             // 3. VERIFICAR SI ALGUNA SESIÓN TUVO 0 ASISTENTES
47             // =====
48
49             for (int i = 0; i < asistencia.Length; i++)
50             {
51                 if (asistencia[i] == 0)
52                 {
53                     existeCero = true;
54                 }
55             }
56
57             if (existeCero)
58             {
59                 Console.WriteLine("\nAlerta: existe una sesión con 0 asistentes");
60             }
61             else
```

```
59     }
60     else
61     {
62         Console.WriteLine("\nNo hay sesión con 0 asistentes");
63     }
64
65     // =====
66     // 4. MODIFICAR UNA ASISTENCIA INDICANDO SU ÍNDICE
67     // =====
68
69     Console.WriteLine("\nIngrese el índice de la sesión que desea modificar: ");
70     indiceModificar = int.Parse(Console.ReadLine());
71
72     if (indiceModificar >= 0 && indiceModificar < asistencia.Length)
73     {
74         Console.WriteLine("Ingrese el nuevo valor de asistencia: ");
75         nuevoValor = int.Parse(Console.ReadLine());
76
77         asistencia[indiceModificar] = nuevoValor;
78     }
79     else
80     {
81         Console.WriteLine("Índice no válido");
82     }
83
84     // =====
85     // 5. MOSTRAR LAS ASISTENCIAS ACTUALIZADAS
86     // =====
87
88     Console.WriteLine("\n=== ASISTENCIAS ACTUALIZADAS ===");
89
90     for (int i = 0; i < asistencia.Length; i++)
```

```
68     Console.WriteLine("\nIngrese el índice de la sesión que desea modificar: ");
69     indiceModificar = int.Parse(Console.ReadLine());
70
71     if (indiceModificar >= 0 && indiceModificar < asistencia.Length)
72     {
73         Console.WriteLine("Ingrese el nuevo valor de asistencia: ");
74         nuevoValor = int.Parse(Console.ReadLine());
75
76         asistencia[indiceModificar] = nuevoValor;
77     }
78     else
79     {
80         Console.WriteLine("Índice no válido");
81     }
82
83     // =====
84     // 5. MOSTRAR LAS ASISTENCIAS ACTUALIZADAS
85     // =====
86
87     Console.WriteLine("\n=== ASISTENCIAS ACTUALIZADAS ===");
88
89     for (int i = 0; i < asistencia.Length; i++)
90     {
91         Console.WriteLine($"Índice {i}: {asistencia[i]} asistentes");
92     }
93
94     Console.ReadKey();
95 }
96
97 }
98
99 }
```

The screenshot shows a Visual Studio IDE with a C# program named 'EJERCICIO_07'. The code in 'Program.cs' includes logic for updating session attendance. The console output shows the program running, with prompts for session index and attendance value, followed by a display of the current attendance list. The user has entered '2' for the session index and '11' for the attendance value, which has been updated in the list.

```
68 Console.WriteLine("Ingrese el índice de la sesión que desea modificar: ");
69 indiceModificar = int.Parse(Console.ReadLine());
70
71 if (indiceModificar >= 0 && indiceModificar < asistencia.Length)
72 {
73     Console.WriteLine("Ingrese el nuevo valor de asistencia: ");
74     nuevoValor = int.Parse(Console.ReadLine());
75     asistencia[indiceModificar] = nuevoValor;
76 }
77 else
78 {
79     Console.WriteLine("Índice no válido");
80 }
81
82 // =====
83 // 5. MOSTRAR LAS ASISTENCIAS ACTUALIZADAS
84 // =====
85 Console.WriteLine("\n=== ASISTENCIAS ACTUALIZADAS ===");
86
87 for (int i = 0; i < asistencia.Length; i++)
88 {
89     Console.WriteLine($"Índice {i}: {asistencia[i]} asistentes");
90 }
91
92 Console.ReadKey();
93
94
95
96
97
```

Console Output:

```
Ingrese la asistencia de la sesión 0: 10
Ingrese la asistencia de la sesión 1: 12
Ingrese la asistencia de la sesión 2: 0
Ingrese la asistencia de la sesión 3: 8
Ingrese la asistencia de la sesión 4: 9

=== ASISTENCIAS REGISTRADAS ===
Índice 0: 10 asistentes
Índice 1: 12 asistentes
Índice 2: 0 asistentes
Índice 3: 8 asistentes
Índice 4: 9 asistentes

Alerta: existe una sesión con 0 asistentes

Ingrese el índice de la sesión que desea modificar: 2
Ingrese el nuevo valor de asistencia: 11

=== ASISTENCIAS ACTUALIZADAS ===
Índice 0: 10 asistentes
Índice 1: 12 asistentes
Índice 2: 11 asistentes
Índice 3: 8 asistentes
Índice 4: 9 asistentes
```

COMANDOS PARA SUBIR AL GITHUB:

git init

git add .

git remote add origin "----" enlace de github

git commit -m "PRACTICA - TAREA - SEMANA 08"

git pull origin master

..... si hay problema entonces se pone

git pull origin master --allow-unrelated-histories

git push -u origin master